

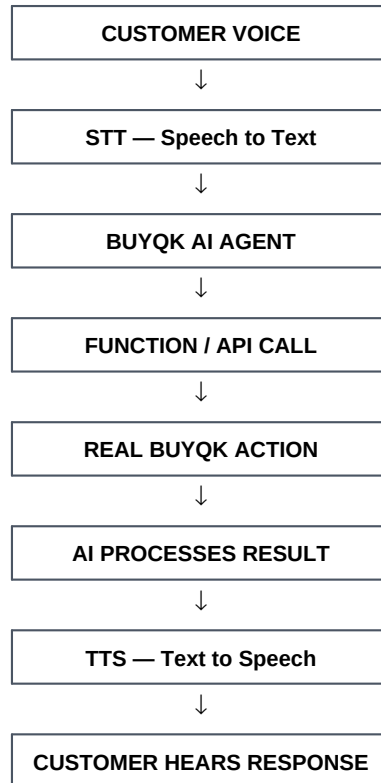
BuyQK Voice AI Agent

Clean Technical Architecture, APIs, Implementation Plan & Timeline

Target: Build an AI shopping agent that can understand → decide → act → confirm → respond through voice.

1. Project Objective

BuyQK should not behave like a voice chatbot that only talks. The system should understand a customer's request, remember the conversation, decide when an action is required, call the correct BuyQK API, process the result, ask for confirmation when needed, and finally respond naturally through TTS.



2. Conversational AI Orchestration

The orchestration layer is the decision-making part of the system. It receives text from STT, checks the conversation state, understands the request, and decides whether to answer directly or call a BuyQK function.

Agent Step	What it does
Understand	Identifies the customer's intent, products, price, size, brand and other details.
Remember	Uses previous messages and current shopping context.
Decide	Determines whether an answer or an action is required.
Act	Selects and calls the correct BuyQK function.
Confirm	Requests confirmation before important actions.
Respond	Turns the final result into a short natural voice response.

3. Function Calling / Action Framework

The AI should not directly modify the BuyQK database. It should call controlled functions exposed by the BuyQK backend. Every function should have a fixed input and output contract.

Function	Purpose	Confirm?
search_products	Search products	No
get_product_details	Get product information	No

Function	Purpose	Confirm?
check_product_availability	Check stock	No
get_cart	View cart	No
add_to_cart	Add product	Usually no
remove_from_cart	Remove product	Usually no
update_cart_quantity	Change quantity	Usually no
apply_coupon	Apply discount	No
get_order_status	Track order	No
get_delivery_estimate	Check delivery	No
place_order	Complete purchase	Yes
cancel_order	Cancel order	Yes
request_refund	Start refund	Yes
change_delivery_address	Change address	Yes

4. Conversation State and Memory

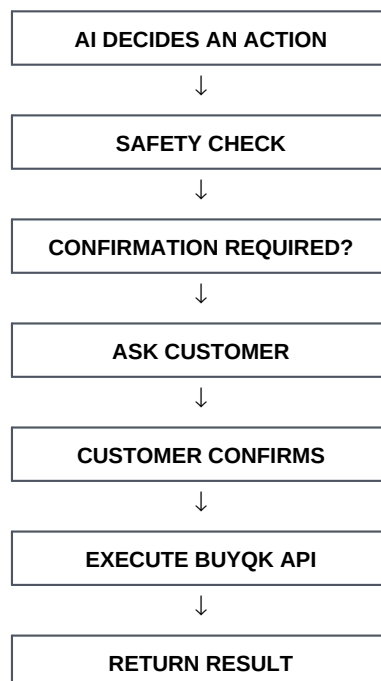
Short-term conversation memory is required so the agent can understand follow-up requests. For example: "Show Nike shoes" → "Only black ones" → "Under \$5,000" should become one combined request.

Session state: session ID, user ID, current intent, current product, cart context, last function, last function result, pending confirmation and recent conversation history.

Optional long-term memory: useful shopping preferences such as preferred brands, sizes and categories.

5. Confirmation and Safety

Important actions must not happen silently. The agent should ask for confirmation before placing an order, cancelling an order, requesting a refund, changing an address or completing a high-value purchase.



6. TTS Provider Strategy — Google Gemini + Groq

Area	Google Gemini	Groq
TTS role	Primary	Secondary / fallback
TTS model	Gemini 2.5 Flash Preview TTS	Orpheus TTS
Strength	Conversational and controllable	Very fast expressive English
Hindi / Hinglish	Better candidate for BuyQK	Needs validation
English	Strong	Strong
STT option	Separate provider	Whisper multilingual
Function calling	Handled by separate AI agent	Handled by text/agent model, not TTS

Google Gemini TTS — Primary

Use Gemini 2.5 Flash Preview TTS as the primary speech path. It is designed for conversational speech and allows control over voice style. It is the better starting point for BuyQK because the product needs English, Hindi and Hinglish conversations.

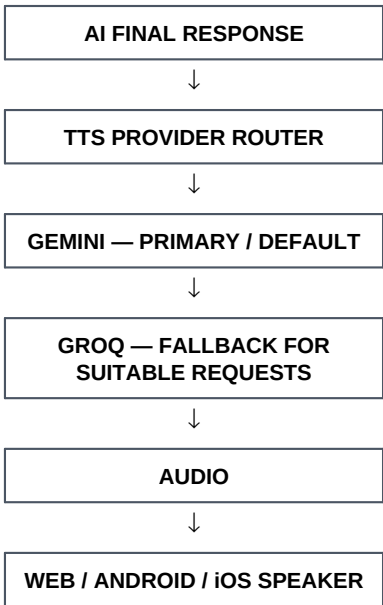
Important architecture point: Gemini 2.5 Flash Preview TTS is a speech model. Its model documentation lists function calling as unsupported. Therefore, the BuyQK AI agent must perform reasoning and function calls separately, and only the final response text should be sent to TTS.

Groq — Secondary / Fallback

Groq provides a fast TTS API and is useful as a secondary provider. Its currently documented TTS models focus on English and Arabic, so it should not be the primary Hindi/Hinglish voice without testing. Groq is also useful for STT because its Whisper models support multilingual transcription.

7. TTS Provider Router

The backend should contain a small provider-router layer. The frontend should not care whether Gemini or Groq generated the audio.



8. Voice Conversation UX

The user should be able to interrupt the AI while it is speaking. When interruption is detected, TTS playback stops, the new speech goes to STT, and the agent processes the new request using the existing conversation context.

Situation	Expected behavior
User interrupts AI	Stop TTS immediately and process new speech.
STT cannot understand	Ask the user to repeat the request.

Situation	Expected behavior
BuyQK API fails	Explain the problem; never invent a result.
TTS fails	Show the text response as a fallback.
Request is unclear	Ask a short clarification question.

9. Real BuyQK Shopping Scenario

Customer: "Mujhe Nike ke running shoes chahiye, 5000 ke andar."

Agent understands: Running shoes + Nike + maximum ₹5,000.

Agent action: Calls `search_products`.

API result: Nike Revolution, ₹4,499, available.

Agent: "I found Nike Revolution running shoes for four thousand four hundred and ninety-nine rupees."

Customer: "Add them to my cart."

Agent: Calls `add_to_cart` and confirms success.

Customer: "Buy them."

Agent: "The total is ₹4,499. Would you like me to place the order?"

Customer: "Yes."

Agent: Calls `place_order`, checks the API result, then says: "Done. Your order has been placed successfully."

10. Required BuyQK API Groups

API group	Examples
Product	<code>/search</code> , <code>/products/{id}</code> , <code>/availability</code>
Cart	<code>/cart</code> , <code>/cart/add</code> , <code>/cart/remove</code> , <code>/cart/update</code>
Order	<code>/orders</code> , <code>/orders/{id}</code> , <code>/orders/{id}/cancel</code> , <code>/orders/{id}/refund</code>
Customer	<code>/customer/profile</code> , <code>/customer/preferences</code> , <code>/customer/addresses</code>

11. Recommended Technology Stack

Layer	Recommended choice
STT	Cloud STT; evaluate Groq Whisper for multilingual input
AI Agent / LLM	BuyQK LLM with tool/function calling
Backend	Python + FastAPI
Conversation State	Redis
Long-term Preferences	BuyQK database
Primary TTS	Google Gemini TTS
Secondary TTS	Groq TTS
API communication	REST
Monitoring	Logs, errors and latency metrics
Clients	Existing Web + Android + iOS apps

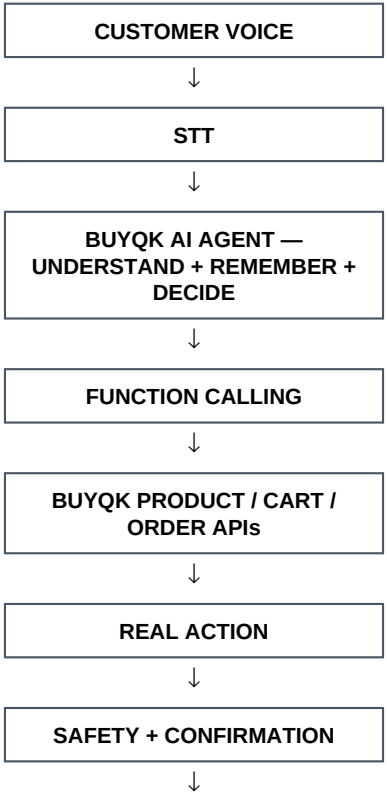
12. Implementation Plan & Timeline

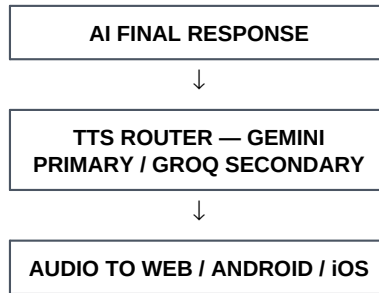
Phase	Work	Time
1	STT, TTS, audio playback and basic errors	3–4 days
2	AI orchestration and conversation state	4–5 days
3	Function calling and BuyQK APIs	5–7 days
4	Confirmation, permissions and safety	2–3 days
5	Barge-in, interruption and fallbacks	3–4 days
6	Gemini + Groq provider routing	2–3 days
7	Real shopping scenarios and edge-case testing	4–5 days
8	Performance, bug fixing and stabilization	3–4 days
	Total MVP	Approximately 4–6 weeks

13. End-to-End Testing

- Search products in English, Hindi and Hinglish.
- Test context: "Show Nike shoes" → "Only black ones" → "Under 5000."
- Test references such as "Add the second one."
- Verify confirmation before order placement.
- Interrupt the AI while TTS is playing.
- Simulate product, cart and order API failures.
- Test unclear speech and STT retry.
- Test Gemini failure and Groq fallback for suitable English requests.
- Verify the agent never claims an action succeeded without a successful API response.

14. Final Architecture





Final recommendation: Keep the AI reasoning and function-calling layer separate from TTS. Use Google Gemini TTS as the primary voice provider and Groq as a secondary/fallback provider. The complete product loop is **Understand** → **Remember** → **Decide** → **Act** → **Confirm** → **Respond**.

Important: Cloud model names, language support, quotas and pricing can change. Provider capabilities should be revalidated before production deployment.